BIT 326 DISTRIBUTED SYSTEMS

Chapter 9 - Consistency and Concurrency Control in Distributed Systems

9.1 Introduction

Consistency is a fundamental property of distributed systems that ensures that all replicas of a shared data store have the same value. Consistency in distributed systems ensures that data, replicated across multiple nodes, appears identical or synchronized to all clients. It governs how reads and writes are ordered and visible. It guarantees that nodes either see the same data at the same time (strong consistency) or converge to the same state eventually (eventual consistency), managing the trade-off between consistency, availability, and performance.

Concurrency control in distributed systems ensures data consistency and transaction serializability when multiple nodes access shared data simultaneously, preventing conflicts like lost updates and dirty reads.

9.2. Consistency Model/ Techniques in Distributed System

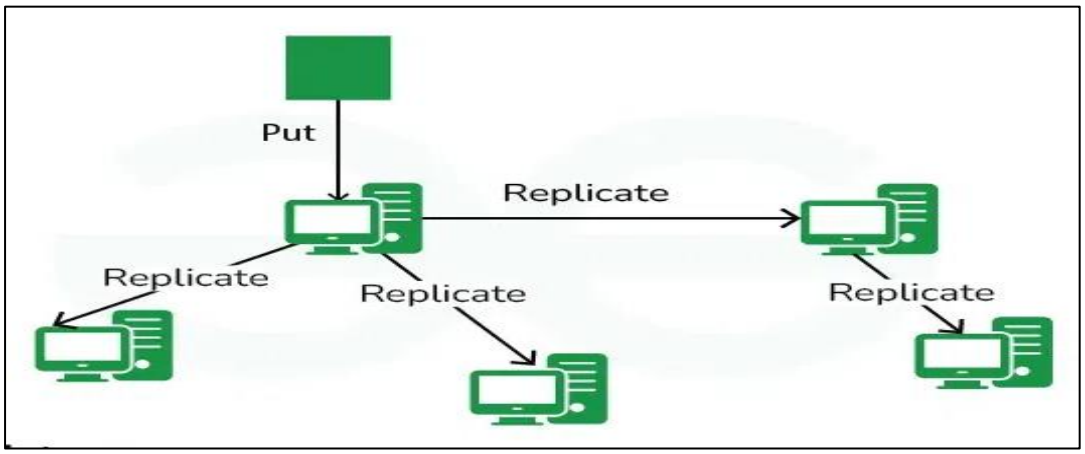
In distributed systems, consistency models define how data updates are shared and viewed across multiple nodes. They set the rules for synchronization, ensuring users and applications interpret data changes correctly. Depending on the approach, strict or relaxed systems balance reliability, availability, and performance.

In distributed Systems, consistency models come in a variety of forms. Every consistency model has advantages and disadvantages, and the system's particular requirements will determine which model is best. Distributed databases use consistency models to define how and when data changes become visible across nodes.

1) **Strong Consistency/Strict Consistency:**
   Strong Consistency ensures that after a write completes, all subsequent reads across any node return the most recent data, creating the illusion of a single-copy system. It prioritizes accuracy over availability and speed, making it essential for financial transactions but causing higher latency.

   Strict consistency (or linearizability) in distributed systems is the strongest models and highest level of data consistency, ensuring that after a write completes, all subsequent reads regardless of which node they access return that value immediately. It behaves as if all operations happen atomically and instantaneously on a single copy of data, ensuring all nodes reflect the same data at the same time. This model makes sure every other server in the system reflects this change immediately. This model provides the highest level of consistency, but it can be slower and require more resources in a distributed environment since all servers must stay perfectly in sync.

   Strong consistency patterns ensure that all replicas of data in a distributed system are updated synchronously and uniformly. Here are a few key patterns:



Key Patterns

a) **Strict Two-Phase Locking:** This pattern employs a locking mechanism to ensure that only one transaction can access a piece of data at a time. It guarantees that transactions are executed in a serializable order, maintaining strong consistency across the system.
b) **Serializability:** Transactions are executed in a manner that preserves the consistency of the system as if they were executed serially, even though they may be executed concurrently. This ensures that the final state of the system is consistent with a sequential execution of transactions.
c) **Quorum Consistency:** In this pattern, a majority of replicas must agree on the value of data before it is considered committed. This ensures that conflicting updates are resolved, and all replicas converge to the same value, maintaining strong consistency.
d) **Synchronous Replication:** All updates to data are synchronously propagated to all replicas before a write operation is considered complete. This ensures that all replicas are always up-to-date and consistent with each other.

2) Eventual Consistency Model

Eventual consistency guarantees that if no new updates are made to the data, all replicas will eventually converge to the same value, even in the presence of network delays or temporary partitions. It prioritizes high availability and low latency over immediate consistency, allowing temporary divergent reads across nodes.

While it may lead to temporary inconsistencies, eventual consistency is suitable for scenarios where immediate consistency is not critical. It is widely used in large-scale systems like DNS and social media

Characteristics of Eventual Consistency

The characteristics of eventual consistency include:

- **Asynchronous Updates:** Updates to the system are not immediately propagated to all nodes. Instead, they are propagated asynchronously, which can lead to temporary inconsistencies between nodes.
- **Lack of Strong Consistency Guarantees:** Eventual consistency does not provide strong guarantees about when updates will be propagated or when all nodes will converge to a consistent state. Instead, it guarantees that given enough time and no further updates, all nodes will eventually converge to a consistent state.
- **Delayed Propagation:** There can be delays in propagating updates across nodes, which can result in temporary inconsistencies where different nodes have different views of the

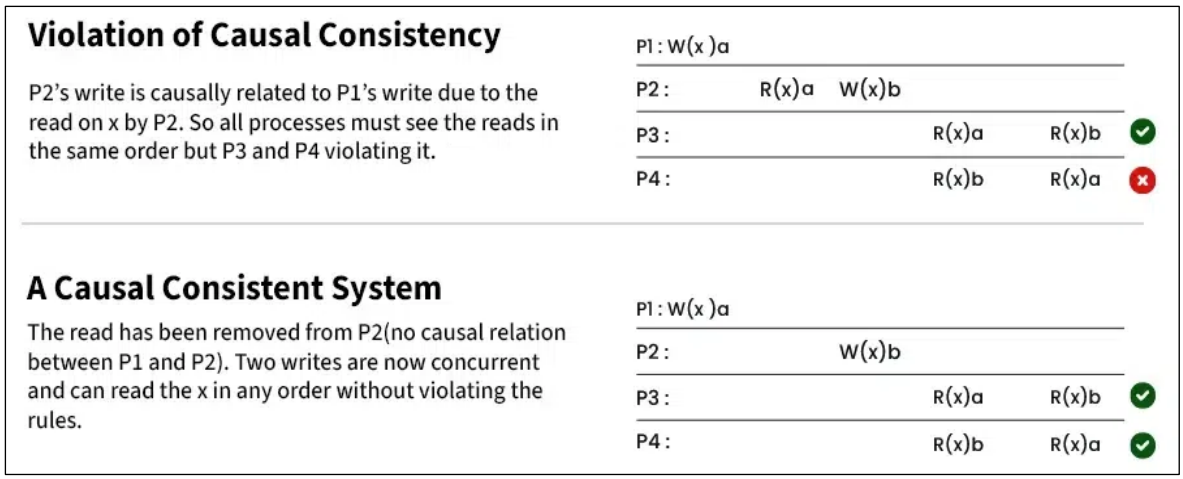
data. These inconsistencies are resolved over time as updates are propagated and applied to all nodes.

- **Conflict Resolution:** In cases where conflicting updates occur (e.g., two nodes update the same data concurrently), eventual consistency requires a conflict resolution mechanism to resolve the conflict and ensure that all nodes eventually converge to a consistent state.
- **Eventual Convergence:** The key characteristic of eventual consistency is that all nodes will eventually converge to a consistent state, given enough time and no further updates. This means that temporary inconsistencies are resolved over time, and the system eventually becomes consistent.

- **High Availability:** Eventual consistency can improve system availability by allowing reads and writes to continue even in the presence of network partitions or node failures. Clients can continue to access the system and perform operations, even if some nodes are temporarily unavailable.

3) **Causal Consistency Model**

The Causal Consistency Model is a type of consistency in distributed systems that ensures that related events happen in a logical order. Causal consistency is a distributed systems model ensures that causally related operations (e.g., a reply to a post) are observed in the same order by all nodes, while independent operations can be seen in different orders. It provides a balance, offering better performance than strong consistency by allowing asynchronous updates while preventing the counter-intuitive ordering issues of pure eventual consistency.



Causal Consistency in Distributed System

Characteristics of Causal Consistency

- Causal Relationship Preservation; Causal consistency ensures that if one event causally influences another, all nodes in the system will observe these events in the same causal order. This means that events that are causally related will be seen in the correct order by all nodes.
- Partial Order- Unlike strong consistency, which enforces a total order of operations, causal consistency allows for a partial order. This means that events that are not causally related can be observed in different orders by different nodes without violating consistency.
- Concurrency and Availability; Causal consistency allows for a higher degree of concurrency compared to strong consistency. This means that operations can be executed in parallel, improving system performance and availability.

- Delayed or Out-of-Order Messages;- Causal consistency handles delayed or out-of-order messages between nodes gracefully. It ensures that even if messages arrive late or in a different order than they were sent, the causal dependencies between events are preserved.
- Trade-offs; - Achieving causal consistency may involve trade-offs in terms of performance and complexity. Implementing causal consistency requires maintaining additional metadata (e.g., vector clocks) to track causal dependencies, which can introduce overhead.

How it works

a) Tracking Causal Relationships: Each operation or event in the system is associated with a timestamp or identifier that indicates when it occurred. This timestamp includes information about the causal dependencies of the operation, such as which other operations it depends on.
b) Ordering Operations: When a node receives a new operation, it checks the timestamp or identifier of the operation to determine its causal dependencies. The node then ensures that the operation is processed after its dependencies have been processed, preserving the causal order.
c) Propagating Updates: As nodes process operations and update their state, they propagate these updates to other nodes in the system. The updates include information about the causal dependencies of the operations, so that other nodes can ensure they process the operations in the correct order.
d) Resolving Conflicts: In cases where there are conflicting operations (i.e., operations that cannot be ordered based on their causal dependencies), the system uses conflict resolution mechanisms to ensure a consistent order. This might involve prioritizing operations based on their timestamps or using other rules to determine the order.
e) Maintaining Consistency: By ensuring that operations are processed in the correct causal order, causal consistency helps maintain a consistent view of the system's state across all nodes.

4) **Sequential Consistency Model:**

It is a consistency model in distributed systems that ensures all operations across processes appear in a single, unified order. In this model, every read and write operation from any process appears to happen in sequence, regardless of where it occurs in the system. Importantly, all processes observe this same sequence of operations, maintaining a sense of consistency and order across the system.

Example

Below is the example of Sequential Consistency in a distributed system with three processes: P1, P2, and P3, interacting with a shared variable X. Below is the example scenario of processes:

- ***P1****: W(X, 5) — Writes the value 5 to X.*
- ***P2****: R(X) — Reads the value of X.*
- ***P3****: W(X, 10) — Writes the value 10 to X.*
- ***P2****: R(X) — Reads the value of X.*

Explanation of Sequential Consistency:

Sequential consistency requires that the result of operations appear as if they were executed in some global sequence that respects the real-time order of operations performed by each process. Possible sequential orders are:

- **Sequential Order 1**:

- **P1**: W(X, 5) — Write 5 to X.
- **P2**: R(X) — Read 5 (as 5 was the last write before this read).
- **P3**: W(X, 10) — Write 10 to X.
- **P2**: R(X) — Read 10 (as 10 was the last write before this read).

Consistency Check:

- The first read by P2 returns 5, which is the value written by P1.
- The second read by P2 returns 10, which is the value written by P3.
- This order maintains sequential consistency as it reflects a global sequence where operations are ordered as if they occurred one after the other.

- **Sequential Order 2**:
  - **P1**: W(X, 5) — Write 5 to X.
  - **P3**: W(X, 10) — Write 10 to X.
  - **P2**: R(X) — Read 10 (as 10 was the latest write before this read).
  - **P2**: R(X) — Read 10 (as 10 was the latest write before this read).

Consistency Check:

- The first read by P2 returns 10, reflecting that it reads the value written by P3, which was the latest write.
- The second read by P2 also returns 10, consistent with the latest write.
- This order is valid because it maintains a consistent view of the latest write operation, reflecting a valid global order.

5) **Session Consistency/ Client Centric:** Session Consistency guarantees that all of the data and actions a user engages with within a single session remain consistent. Session consistency is a client-centric model in distributed systems ensuring that within a user's session, read and write operations remain consistent and ordered, even if the system is not globally consistent. It guarantees that users see their own writes immediately (Read-Your-Writes) and experience data in chronological order (Monotonic Reads), balancing high availability with a logical user experience.

Components

- Read Your Writes (RYW): If a user updates data, any subsequent read by the *same* user within that session will return the updated data. This led to
  a) **Enhanced User Experience:** By guaranteeing that subsequent reads reflect recent writes, the system becomes more predictable and user-friendly, reducing confusion and frustration.
  b) **Data Integrity and Reliability:** Ensuring that users can see their own recent changes prevents discrepancies in data, maintaining the integrity of the information presented to them.
  c) **Simplified Development and Maintenance:** Immediate visibility of writes reduces the chances of bugs related to stale data, making the system more robust and easier to maintain.
  d) **Critical for Real-Time Applications:** In environments like collaborative editing or shared workspaces, seeing immediate changes is critical for efficient collaboration and productivity.
  e) Consistency in Distributed Systems; In distributed systems, maintaining read-your-writes consistency ensures that users have a coherent view of their data across different sessions and devices
- Monotonic Reads: Once a user reads a value, they will never see an older value in subsequent reads, ensuring consistent progress.

- Write Follow Reads (WFR): Writes performed by a session after a read are ordered after that read.
- Monotonic Writes: Updates from a session are applied in the order they were submitted

6) **Weak Consistency Model**

A Weak Consistency model in distributed systems prioritizes high availability and low latency over immediate data consistency. It provides no guarantees about the ordering of operations or the state of the data at any given time. Clients may see different versions of the data depending on which node they connect to. This model provides the highest availability and scalability but at the cost of consistency.

Key Characteristics

- **No Immediate Guarantees:** When a write occurs, there is no assurance that subsequent reads from different nodes will reflect that update immediately.
- **Delayed Propagation:** Updates are propagated asynchronously, often causing replicas to lag or diverge for extended periods.
- **High Scalability/Availability:** By removing the need for strict synchronization between nodes, systems can handle higher loads and remain available even during network partitions.
- **Stale Reads:** Applications must be designed to tolerate reading outdated data.
- **Eventual Consistency:** Weak consistency models often holds eventual consistency, where updates made to the system will eventually spread to all nodes and all nodes will meet to a consistent state given enough time and absence of new updates.

7) **Monotonic Reads and Writes:** Consistency models in distributed systems that ensure a predictable, non-decreasing order of operations for a single client. Monotonic reads guarantee that a user never sees older data after having seen newer data, even if reading from different replicas. Monotonic writes guarantee that a process's writes are applied in the order they were submitted. Once a piece of data has been read or written, monotonic reads and writes guarantee that the data will always be viewed in a predictable and consistent order in subsequent reads or writes.

- **Monotonic Reads**: If you read a value from a system, the next time you read it, you will either get the same value or a more recent one. You won't get an older value after seeing a newer one.

  Purpose: Prevents "traveling back in time" scenarios where a user reads updated data and then a later read shows older data due to replication lag.

  Example: When reading email, if you see a new message, a refresh will not make that message disappear
- **Monotonic Writes**: This ensures that once a write happens, all future writes will follow in the correct order. If you update a record or send a message, the system guarantees that it won't reverse the order of your updates.

  Purpose: Ensures that updates from a client are applied in the correct order, which is essential for data integrity.

  Example: If you update a document, the system ensures your first edit is applied before your second edit, preventing newer changes from being overwritten by older ones.

8) **Data-Centric Consistency Models**

- Strict Consistency: The strongest model; absolute, real-time ordering of operations, essentially implying zero latency between a write and a read.

- FIFO (First-In-First-Out): Writes by a single process are observed in the order they were issued.
- Weak Consistency: Does not guarantee that writes are visible immediately, requiring specific synchronization points.
- Release Consistency: Uses special acquire and release operations to manage shared memory, similar to locking mechanisms.

9.3. When to choose which Consistency Model?

1) **Strong Consistency:** Every read reflects the latest write. Use it when the most recent data is critical, like banking or inventory systems.
2) **Sequential Consistency:** Operations appear in a predictable order, but not globally synchronized. Useful when order matters but full synchronization isn't needed.
3) **Causal Consistency:** Related operations are seen in the correct order, but unrelated operations may vary. Ideal for social media or user interaction systems.
4) **Weak Consistency:** Data may be temporarily out of sync, prioritizing availability and performance. Suitable when real-time consistency isn't essential.
5) **Session Consistency:** Data is consistent within a user session but may differ across sessions. Good for applications where session-level consistency matters.
6) **Monotonic Reads/Writes:** User's view of data never goes backward in time. Ensures sequential actions remain consistent and avoids outdated information

9.4. The challenges of consistency in Distributed Systems

Distributed systems operate across multiple nodes over unreliable networks, introducing challenges such as:

- Network failures: Messages can be delayed, dropped, or arrive out of order, leading to inconsistencies between nodes.
- Node crashes: Nodes may fail unexpectedly, potentially losing recent updates before recovering.
- Latency: Communication delays can cause nodes to temporarily hold different versions of the same data, affecting real-time consistency.

9.5. The CAP Theorem and BASE Model

The CAP Theorem and BASE Consistency Model are key principles in distributed system. The CAP Theorem highlights the trade-offs between Consistency, Availability, and Partition Tolerance, suggesting a system can prioritize only two. The BASE Model (Basically Available, Soft state, Eventual consistency) offers a more flexible approach, focusing on eventual consistency over strong guarantees, making it ideal for highly available, scalable systems.

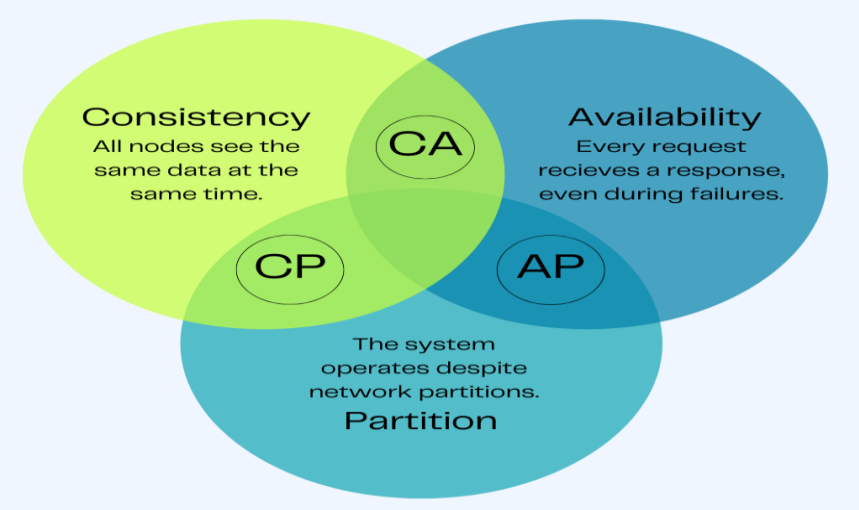
CAP Theorem

According to the CAP theorem, only two of the three desirable characteristics; Consistency (C) (all nodes see the same data simultaneously), Availability (A) (every request gets a response), and Partition Tolerance (P) (system works despite network failures) can be shared or present in a networked shared-data system or distributed system.

- The theorem provides a way of thinking about the trade-offs involved in designing and building distributed systems.

- It helps to explain why certain types of systems may be more appropriate for certain use cases.
- The theorem states that a distributed system can have at most two of these guarantees.

Because network partitions are inevitable, systems must choose between consistency (CP) or availability (AP)



In the context of CAP, consistency refers to strong, atomic, or linearizable consistency, where all nodes return the most recent write, ensuring every client has the same view of the data.

- **Trade-off (CP vs. AP):** When a network partition occurs, a system must either block requests (sacrificing Availability for Consistency) or allow potentially stale reads (sacrificing Consistency for Availability).
- **CP Systems (Consistency & Partition Tolerance):** Prioritize data accuracy over uptime. If nodes cannot communicate, the system returns an error rather than inconsistent data (e.g., PostgreSQL, MongoDB, Zookeeper).
- **AP Systems (Availability & Partition Tolerance):** Prioritize uptime, guaranteeing that all requests are answered, though they might return stale data (e.g., Cassandra, DynamoDB, CouchDB).

Advantages of CAP Theorem:

- CAP forces clarity on system design, making engineers choose which properties to prioritize.
- Allows systems to prioritize consistency, availability, or partition tolerance based on their use case.

Disadvantages of CAP Theorem:

- You can't have all three properties, which can make system design more difficult.
- You may have to sacrifice availability for consistency, or vice versa, based on system priorities.

Features of CAP Theorem:

- Consistency, availability, and partition tolerance are mutually exclusive.
- Forces a choice between strong consistency or high availability.
- Prioritizes handling network partitions (fault-tolerant systems).
- Relevant to both small-scale and large-scale distributed systems.

BASE Consistency Model

BASE stands for Basically Available, Soft state, and Eventually consistent. This model is an alternative to the strict ACID model (used in traditional databases). It is better suited for large distributed systems where availability and performance are more critical than strong consistency.

- Advantages of BASE Consistency Model:
  - BASE systems are designed to handle large amounts of data and users, making them ideal for distributed systems like NoSQL databases.
  - BASE focuses on availability, ensuring the system responds to requests even when it's not fully consistent.

- Disadvantages of BASE Consistency Model:
  - The system may show stale data for some time, which can be problematic for applications needing immediate consistency.
  - BASE models require developers to think carefully about how and when consistency is reached, which can complicate system design.

- Features of BASE Consistency Model:
  - Focuses on high availability and scalability.
  - Allows temporary inconsistencies but guarantees eventual consistency.
  - Adaptable to dynamic environments with fluctuating traffic.
  - Designed for systems where performance is more important than strong consistency.

Differences between CAP Theorem and BASE Consistency Model

Below the difference between CAP Theorem and BASE Consistency model:

Aspect	**CAP Theorem**	**BASE Consistency Model**
Focus	Consistency, Availability, and Partition Tolerance	Availability and Eventual Consistency
Consistency Model	Strong Consistency (requires consistent state after every operation)	Eventual Consistency (system becomes consistent over time)
Availability	Can prioritize either availability or consistency	Prioritizes availability over consistency
Partition Tolerance	Must tolerate network partitions (distributed nature)	Assumes network partitions and operates accordingly
Trade-offs	Forces a trade-off between Consistency and Availability	Allows for partial inconsistency temporarily
Use Case	Ideal for systems needing strict consistency (e.g., banking)	Suitable for large-scale, high-availability systems (e.g., social media)
Complexity	Higher complexity due to consistency requirements	More flexible, simpler handling of temporary inconsistencies
Examples	RDBMS, transactional systems	NoSQL databases, distributed caching systems

9.6. Concurrency Control in Distributed Transactions

Concurrency control ensures that multiple transactions running concurrently across different sites do not lead to data inconsistency or conflicts. Concurrency control is a fundamental concept in distributed / database systems that ensures correct execution of simultaneous transactions without violating data integrity. Below are some characteristics of concurrency control:

a) It enforces isolation among transactions.
b) It preserve database consistency through consistency preserving execution of transactions.
c) It resolve read-write and write-read conflicts.

Concurrency control in distributed systems manages simultaneous access to shared data across multiple independent nodes to ensure data consistency, prevent lost updates, conflicts (like write-write conflicts), and maintain system integrity. It ensures that interleaved operations from different users or services do not interfere, typically using techniques like locking (pessimistic), timestamp ordering, or conflict detection (optimistic).

9.7. Core Concurrency Problems in Distributed Systems

Concurrency problems in distributed systems arise when multiple nodes or processes access and modify shared data simultaneously, leading to data inconsistency, corruption, and race conditions. Key issues include lost updates, dirty reads, deadlocks, and inconsistent views of data across nodes, usually resolved by optimistic/pessimistic concurrency controls, serialization, and locking.

1) **Lost Update Problem:** Occurs when two or more transactions (A and B) update the same data simultaneously, and one transaction overwrites the other, causing updates to be lost (a write-write conflict).
2) **Dirty Read Problems:** A transaction reads data that has been updated by another transaction but not yet committed. If the first transaction rolls back, the second transaction has read invalid data.
3) **Incorrect Summary/Inconsistency Problem:** Occurs when an aggregated read operation (e.g., sum) occurs while other transactions are writing, leading to an inconsistent view where some data is updated and some is not.
4) **Unrepeatable read Problem:** A transaction reads the same row multiple times but receives different data because another transaction modified it between reads.
5) **Phantom Reads:** A transaction executes a query returning a set of rows, but a subsequent execution of the same query returns a different set because another transaction inserted or deleted rows.
6) **Deadlocks:** In pessimistic systems, multiple transactions lock resources in an incompatible order, resulting in a stalemate where no transaction can proceed.

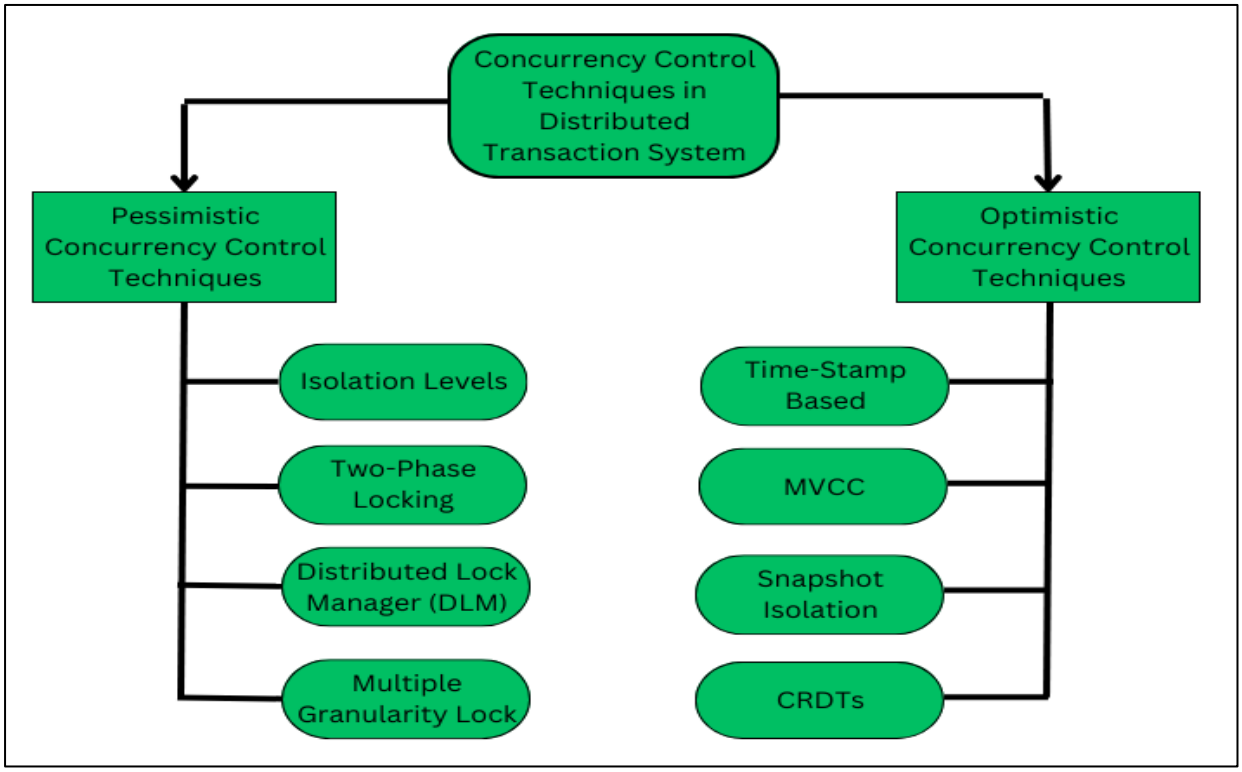
9.7.1. Causes and Challenges

1) **Data Partitioning/Replication:** Managing data across multiple nodes means ensuring consistency across distributed locations is challenging.
2) **Network Latency & Failure:** Communication delays and node failures make achieving global consensus on order challenging.
3) **Clock Synchronization:** Differences in time across machines make it difficult to determine the exact order of events

9.8. Concurrency Control in Distributed systems Types

Classified as;

1) Pessimistic Concurrency Control Techniques (PCC)
2) Optimistic Concurrency Control Techniques (OCC)



1) Pessimistic Concurrency Control (PCC): a technique used in database systems to prevent conflicts between concurrent transactions by locking data before it is accessed. It assumes that conflicts are likely to occur, so it blocks other transactions from accessing the data until the current transaction is complete. This approach ensures high data consistency by preventing issues like dirty reads, lost updates, and unrepeatable reads, but may lead to reduced performance due to blocking and waiting.

   Pessimistic Concurrency Control Methods

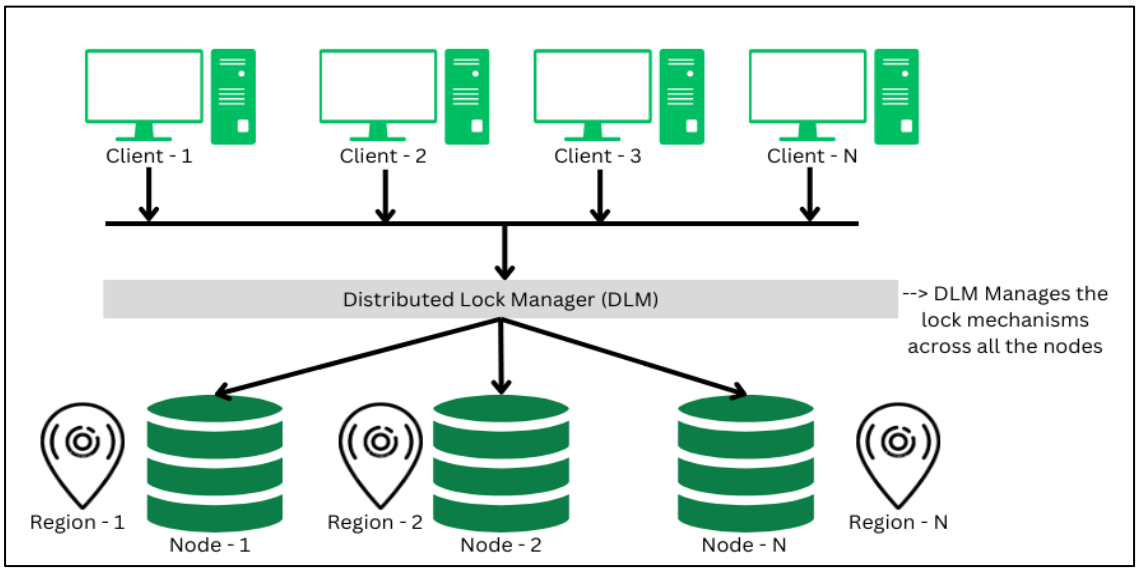
   a) Isolation Level: Isolation levels define how strictly transactions are kept isolated from each other to prevent data inconsistencies. Without proper isolation, one transaction may read or modify data while another is still processing it, leading to conflicts. The four standard isolation levels are:
      - Read Uncommitted
      - Read Committed
      - Repeatable Read
      - Serializable

      Each level offers a different balance between concurrency and consistency.

   b) Two-Phase Locking Protocol: The two-phase locking protocol is a concurrency technique used to manage locks on data items in database. This technique consists of 2 phases:
      - *Growing Phase*: The transaction acquires all the locks on the data items that'll be required to execute the transaction successfully. No locks will be realease in this phase.

- *Shrinking Phase*: All the locks acquired in previous phase will be released one by one and No New locks will be acquired in this phase.

c) Distributed Lock Manager: A distributed lock a critical component in the distributed transaction system, which co-ordinates the lock acquiring, and releasing operations in the transactions. It helps in synchronizing the transaction and their operation so that data integrity is maintained.



Distributed Lock Manager (DLM)

d) Multiple Granularity Lock: A lock can be acquired at various granular level like: table level, row/record level, page level or any other resource's level. In transaction system a transaction can lock a whole table, or a specific row while performing some changes on it. This lock acquiring when done by various transactions simultaneously, this phenomena is called as multiple granularity locking.

2) Optimistic Concurrency Control (OCC): The Optimistic Concurrency control techniques proceeds on the basis of assumption that, 0 or very few transactions will try to access a certain resource simultaneously. We can describe a system as FULLY OPTIMISTIC, if it uses No-Locks at all & checks for conflicts at commit time. It has following 4-phases of operation:

a) Read Phase: When a transaction begins, it reads the data while also logging the time-stamp at which data is read to verify for conflicts during the validation phase.

b) Execution Phase: In this phase, the transaction executes all its operation like create, read, update or delete etc.

c) Validation Phase: Before committing a transaction, a validation check is performed to ensure consistency by checking the last_updated timestamp with the one recorded at read_phase. If the timestamp matches, then the transaction will be allowed to be committed and hence proceed with the commit phase.

d) Commit phase: During this phase, the transactions will either be committed or aborted, depending on the validation check performed during previous phase. If the timestamp matches, then transactions are committed else they're aborted.

Optimistic Concurrency Control Methods

a) Timestamp Based (OCC): In timestamp-based concurrency control, each transaction gets a unique timestamp when it starts. At commit time, if the data was modified by a newer

transaction, the current transaction is aborted or restarted based on system policy. If no conflicting updates occurred, the transaction is committed.

Example: Let's say we have two transaction T1 and T2, they operate on data item - A. The Timestamp concurrency technique will keep track of the timestamp when the data was accessed by transaction T1 first time.

Transaction	**Data item and operation**	**Most_recent_Timestamp**	**Initial_timestamp of data item (A)**
T1	Read(A)	12:00PM	12:00PM
T2	Write(A)	12:15PM	12:00PM
T1	Write(A)	12:30PM	12:00PM

Now, let's say this transaction T1 is about to commit, before committing, it will check the initial timestamp with the most recent timestamp. In our case, the transaction T1 won't be committed because a write operations by transaction T2 was performed.

```
if(Initial_timestamp == Most_recent_timestamp)
then 'Commit'
else
'Abort'
```

In our case, transaction will be aborted because T2 modified the same data item at 12:15PM.

b) Multi-Version Concurrency Control (MVCC): In distributed systems MVCC improves performance by maintaining multiple versions of data items, allowing read-only transactions to read consistent, older snapshots without locking. It enables high parallelism, as reads do not block writes and vice versa, using timestamps to provide consistent, snapshot-isolated views of data across distributed nodes. In MVCC, every data item has multiple versions of itself. When a transaction starts, it reads the version that is valid at the start of the transaction. And when the transaction writes, it creates a new version of that specific data item. That way, every transaction can concurrently perform their operations.

Key Aspects of Distributed MVCC:

- **Version Creation:** Instead of updating in-place, writes create a new version of the data item.
- **Snapshot Isolation:** Transactions read from a specific, stable version (snapshot) based on their start timestamp, ensuring consistent reads.
- **Conflict Handling:** Conflicts are generally handled by allowing reads to access older data while a newer version is being written.
- **Timestamp Ordering:** Transactions are typically assigned unique timestamps (e.g., from a central or distributed source) to determine which version is visible.
- **Garbage Collection:** Old, obsolete versions must be removed periodically to prevent storage bloat.

Types

- **Timestamp-Based MVCC:** Each transaction is given a **unique timestamp**. This timestamp decides which version of a record the transaction is allowed to read or update.
  A newer transaction will only see versions created **before** its timestamp.
  Use-case: Systems that need strict ordering of transactions.
- **Snapshot-Based MVCC:** When a transaction starts, the database gives it a **snapshot** of the entire data at that moment. The transaction works on this stable snapshot even if other transactions update data in the meantime.
  Use-case: Databases like PostgreSQL that want consistent reads without locking.
- **History-Based MVCC:** The database keeps a **full history** of all versions of a record. This makes rollbacks easier because the system can simply move back to an earlier version.
  Use-case: Systems requiring detailed version tracking or auditing.
- **Hybrid MVCC:** This combines two or more MVCC techniques to balance:
  - Performance
  - Storage usage
  - Version visibility

  Use-case: Modern DBMS optimizing for mixed read/write workloads.

How MVCC Works

- **Each record has a version number** that shows which version is the latest.
- **Read operations always use the newest committed version**, so multiple users can read the same data at the same time without waiting.
- **Write operations don't overwrite the existing record.** Instead, the system creates a **new copy** of the record.
- While a new version is being created, other users can still read the old version without interruption.
- After the write is successfully completed, the version number increases, and the new version becomes the current one.
- Every future read or update uses this latest version, and the same cycle continues whenever updates happen.

c) Snapshot Isolation: Snapshot isolation is basically a snapshot stored in an isolated manner when our database system was purely consistent. And this snapshot is read by the transactions at the beginning. Transaction ensures that the data item is not changed while it was executing operations on it. Snapshot isolation is achieved through OCC & MVCC techniques.

- Conflict Free Replicated Data Types (CRDTs): CRDTs is a data structure technique which allows a transaction to perform all its operation and replicate the data to some other node or current node. After all the operations are performed, this technique offers us with merging methods that allows us to merge the data across distributed nodes (conflict-free) and eventually achieving consistent state (eventually consistent property).

3) **Wait-Die (WD) algorithm**. In a distributed system, resources are spread across multiple nodes. The Wait-Die algorithm operates without a central, global controller, as the decision is based solely on the timestamps of the involved transactions. It is a non-preemptive deadlock

prevention technique used in distributed systems, specifically designed to handle deadlocks in distributed databases and concurrency control. It uses transaction timestamps to prioritize older transactions over younger ones to prevent circular wait conditions.

- **Deadlock Prevention:** By enforcing a strict ordering of transactions (older wait, younger die), the circular wait condition necessary for deadlocks is never met.
- **Performance Implications:** A major disadvantage is the potential for unnecessary rollbacks, as a young process might die even if the older process was not actually in a deadlock state.
- **Timestamp-Based:** When a transaction T_i starts, it is assigned a unique global timestamp $TS(T_i)$. A smaller timestamp indicates an older transaction, while a larger one indicates a younger transaction.
- **Non-Preemptive:** A process is not forcefully preempted (killed) by another; instead, it is allowed to "die" (abort) itself if it cannot wait.
- **The Rule:** If transaction T_i requests a resource held by T_j;
  a) **Wait:** If $TS(T_i) < TS(T_j)$ (T_i is **older** than T_j), T_i is allowed to wait
  b) **Die:** If $TS(T_i) > TS(T_j)$ (T_i is **younger** than T_j), T_i dies (aborts and restarts with the same timestamp)
- **Starvation Prevention:** Because a dying process restarts with the same, original (old) timestamp, it eventually becomes the oldest process in the system, preventing starvation.

Comparison with Wound-Wait (WW)

Wait-Die is often contrasted with the **Wound-Wait** algorithm, which is pre-emptive.

Feature	Wait-Die (WD)	Wound-Wait (WW)
Approach	Non pre-emptive	Pre-emptive
Old asks Young	Old waits	Old wounds (pre-empts) Young
Young asks Old	Young dies (aborts)	Young waits
Rollbacks	Higher (young dies)	Lower

Summary of Workflow

1. **Request:** Process P_1 requests a resource held by P_2
2. **Compare:** If P_1 is older, P_1 waits.
3. **Result:** If P_1 is younger, P_1 rolls back and waits to retry

9.9. Key Techniques for Distributed Concurrency

1) Two-Phase Locking (2PL): Transactions acquire locks (shared or exclusive) on data items before reading or writing, releasing them only upon completion to ensure isolation.
2) Timestamp Ordering**:** Each transaction receives a unique timestamp, and operations are ordered based on these times, effectively preventing deadlocks.
3) Optimistic Concurrency Control**:** Assumes conflicts are rare; transactions proceed without locking but check for conflicts at commit time, aborting if conflicts exist.
4) Conflict-free Replicated Data Types (CRDTs)**:** Allows local updates without locking, merging data later to achieve consistency without conflicts.

9.10. Challenges and Considerations

1) **Data Inconsistency:** Simultaneous reads/writes to the same resource across different machines can lead to duplicate records or overwritten data.
2) **Deadlock Management:** Ensuring that resources locked by different nodes do not result in circular waits.
3) **Performance vs. Consistency:** High consistency often reduces performance (latency), requiring a balance tailored to the application needs.
4) **Clock Synchronization:** Timestamp-based methods require synchronized clocks across all distributed nodes.